C Shim Architecture

Existence Proof and Functional Audit

Date	2026-03-09
Scope	`src/zenoh_dart.{h,c}` — 62 exported symbols, ~700 lines
Reviewed against	zenoh-c v1.7.2, Dart SDK 3.11.0 (FVM), ffigen v20.1.1
Audience	Expert C and Dart code reviewers
Supersedes	`CA-C_shim_audit.md`, `CA-C_shim_proof.md`, `CC-C_shim_audit.md`

zenoh-dart Project

All Dart API documentation references verified against Dart 3.11.1 (latest stable).

Contents

I Why the C Shim Must Exist 1

1 How Dart FFI Resolves Symbols 1
- 1.1 Dart FFI Is C-Only . . . 1
- 1.2 Symbol Resolution via `dlsym` . . . 1
- 1.3 Confirmed by ffigen Bug Tracker . . . 2
- 1.4 How the Shim Bridges This . . . 2

2 Pattern 1: `static inline` Move Functions 2
- 2.1 The Problem . . . 2
- 2.2 Symbol Table Proof . . . 3
- 2.3 Why These Are Required . . . 3
- 2.4 What the Moved Types Actually Are . . . 4
- 2.5 How the C Shim Solves This . . . 5

3 Pattern 2: C11 `_Generic` Polymorphic Macros 5
- 3.1 The Problem . . . 5
- 3.2 Compounding With Pattern 1 . . . 6

4 Pattern 3: Options Struct Initialization 6
- 4.1 The Problem . . . 6
- 4.2 The Struct Layout Problem . . . 6
- 4.3 How the C Shim Solves This . . . 7

5 Pattern 4: Opaque Type Sizes 7
- 5.1 The Problem . . . 7
- 5.2 How the C Shim Solves This . . . 8

6 Pattern 5: Closure Callbacks Across Thread Boundaries 8
- 6.1 The Problem . . . 8
- 6.2 Why Dart FFI Cannot Construct These Closures . . . 8
  - 6.2.1 Constraint A: No closures in `Pointer.fromFunction` . . . 8
  - 6.2.2 Constraint B: Non-Dart thread invocation is undefined behavior . . . 9
  - 6.2.3 Constraint C: `NativeCallable.listener` limitations . . . 9
- 6.3 How the C Shim Solves This . . . 9

7 Pattern 6: Loaning and Const/Mut Enforcement 10
- 7.1 The Problem . . . 10
- 7.2 Why the Per-Type Functions ARE Exported . . . 10
- 7.3 Barrier A: Macro Dispatch . . . 10
- 7.4 Barrier B: Const Qualification Erasure . . . 10
- 7.5 How the Shim Resolves This . . . 11
- 7.6 Why This Is Distinct From Patterns 1 and 2 . . . 11

8 End-to-End Worked Example: `z_put` 11
- 8.1 What a C Consumer Does . . . 11
- 8.2 What Dart FFI Cannot Do . . . 12
- 8.3 The Full Call Chain . . . 12
- 8.4 What the Shim Does NOT Do . . . 12

II Function-by-Function Analysis 13

9 Initialization (2 functions) 13

10 Config (5 functions) 13

11 Session (4 functions) 13

12 KeyExpr (4), Bytes (6), String/View String (7) 13

13 Put / Delete (2 functions) 14

14 Subscriber (3 functions + 2 static callbacks) 14

15 Publisher (8 functions + 2 static callbacks) 14

16 Info / ZID (4 functions) 14

17 Scout (2 functions + 1 static callback) 14

18 Shared Memory (13 functions, conditionally compiled) 14

III Cross-Cutting Analysis 15

19 Memory Ownership Protocol 15

20 Thread Safety 15

21 Findings 15

IV Pattern Resolution Matrix 17

V Conformance Summary 19

22 C Best Practices 19

23 Dart FFI Best Practices 19

24 Architecture Conformance 19

VI Pattern Summary 20

Appendices 21

A Findings Summary 21

B References 21

B.1 Dart FFI Official Documentation 21

B.2 Bug Tracker Citations 21

B.3 zenoh-c Source Files 22

B.4 ISO C Standards 22

B.5 Symbol Table Verification 22

Part I
Why the C Shim Must Exist

The zenoh-c v1.7.2 public API relies on six patterns that Dart FFI cannot call directly. This part proves each pattern exists in the zenoh-c headers with exact file references, explains why Dart FFI cannot handle it with references to official Dart documentation, and shows how the C shim resolves each one.

1 How Dart FFI Resolves Symbols

1.1 Dart FFI Is C-Only

The `dart:ffi` library documentation (Dart 3.11.1) states:

> *"Foreign Function Interface for interoperability with the C programming language."*
> — *dart:ffi library overview*

> *"Dart mobile, command-line, and server apps running on the Dart Native platform can use the dart:ffi library to call native C APIs, and to read, write, allocate, and deallocate native memory."*
> — *C interop guide*

The binding generator (`ffigen` v20.1.1) reinforces this constraint:

> *"Note: FFIgen only supports parsing `C` headers, not `C++` headers."*
> — *ffigen package*

No Dart 3.x release has added support for calling C macros, `static inline` functions, or C++ from `dart:ffi`. The Dart what's new page for releases 3.0 through 3.11 contains no FFI changes related to these constructs. Dart "build hooks" (formerly "native assets") change how native libraries are *built and bundled* but do not alter how `dart:ffi` resolves symbols at runtime.

1.2 Symbol Resolution via `dlsym`

`DynamicLibrary.open` loads a shared library and provides access to its symbols:

> *"A dynamically loaded library is a mapping from symbols to memory addresses."*
> — *DynamicLibrary class*

`DynamicLibrary.lookup` resolves a symbol name to a memory address — functionally equivalent to POSIX `dlsym(3)`:

> *"Looks up a symbol in the DynamicLibrary and returns its address in memory."*
> *"Similar to the functionality of the dlsym(3) system call."*
> *"The symbol must be provided by the dynamic library."*
> — *DynamicLibrary.lookup*

Consequence: Only symbols present in the `.so`'s dynamic symbol table (`nm -D`) are accessible. The following C constructs produce **no symbols**:

Construct	Why No Symbol
`#define` macros	Text substitution by the preprocessor. No compiled code.
`static inline` functions	Internal linkage + inlined. Not exported.
`_Generic` macros	Compile-time type dispatch. Preprocessor construct.
C++ overloaded functions	Name-mangled. No stable C ABI symbol.

Table 1: C constructs that produce no dynamic symbols

1.3 Confirmed by ffigen Bug Tracker

The `ffigen` maintainers have explicitly addressed `static inline` functions in two GitHub issues:

Issue #146 ("Unavailable inline C function"): When `ffigen` previously generated bindings for inline functions like `static inline uint16_t libusb_cpu_to_le16(...)`, the generated Dart code referenced a dynamic library symbol that did not exist. The team's resolution (merged in ffigen 2.0.1) was to **filter out inline functions** entirely:

> *"We should skip them. (Generating an implementation for an arbitrary function would be very involved.)"*
> — *dart-lang/ffigen#146, closed as COMPLETED*

Issue #459 ("Inline function available in compiled dylib, but no bindings generated"): Even when functions are marked `extern inline` (making them available in the `.so`), `ffigen` initially refused to generate bindings. A partial fix (PR #594) was merged to allow `extern inline` functions, but `static inline` functions remain **permanently excluded** — they are not symbols, so there is nothing to bind to.

> *"Static inline functions don't appear in compiled dylibs and cannot be bound — this behavior is expected."*
> — *dart-lang/native#459*

This confirms that `static inline` is a known, permanent limitation of Dart FFI, not an oversight that might be fixed in a future Dart release.

1.4 How the Shim Bridges This

The C shim (`src/zenoh_dart.c`) is compiled as a C translation unit that `#includes` the zenoh-c headers. During compilation, the C compiler:

1. Expands all `#define` macros
2. Inlines all `static inline` functions
3. Resolves all `_Generic` dispatches

The shim's functions are marked `FFI_PLUGIN_EXPORT` (resolving to `__attribute__((visibility("default")))`), which ensures they appear in `libzenoh_dart.so`'s dynamic symbol table. Dart FFI can then call them via `DynamicLibrary.lookup`.

2 Pattern 1: `static inline` Move Functions

2.1 The Problem

zenoh-c uses an ownership model where consuming a resource requires converting a `z_owned_*_t*` pointer to a `z_moved_*_t*` pointer. This conversion is done via `static inline` functions defined in `extern/zenoh-c/include/zenoh_macros.h`.

There are 56 such functions. Here are the ones used by the C shim:

Listing 1: Move functions from `zenoh_macros.h` (lines 7–51)

```c
static inline z_moved_bytes_t* z_bytes_move(z_owned_bytes_t* x) {
   return (z_moved_bytes_t*)(x);
}
static inline z_moved_config_t* z_config_move(z_owned_config_t* x) {
   return (z_moved_config_t*)(x);
}
static inline z_moved_session_t* z_session_move(z_owned_session_t* x) {
   return (z_moved_session_t*)(x);
}
static inline z_moved_publisher_t* z_publisher_move(z_owned_publisher_t* x) {
   return (z_moved_publisher_t*)(x);
}
static inline z_moved_subscriber_t* z_subscriber_move(z_owned_subscriber_t* x) {
   return (z_moved_subscriber_t*)(x);
}
static inline z_moved_encoding_t* z_encoding_move(z_owned_encoding_t* x) {
   return (z_moved_encoding_t*)(x);
}
// ... 50 more variants for other types
```

These are pointer casts at the C level — they reinterpret the same memory address as a different pointer type. But they are `static inline`:

- `static` — internal linkage (not visible outside the translation unit)
- `inline` — the compiler may substitute the function body at the call site

Neither property produces an exported symbol.

2.2 Symbol Table Proof

Running `nm -D` on the compiled `libzenohc.so` confirms these symbols do not exist:

```
$ nm -D libzenohc.so | grep z_bytes_move
(no output)

$ nm -D libzenohc.so | grep z_config_move
(no output)

$ nm -D libzenohc.so | grep z_session_move
(no output)
```

Zero results. These functions exist only in the header file. Dart FFI's `DynamicLibrary.lookup('z_bytes_move')` would throw an `ArgumentError`.

2.3 Why These Are Required

The move functions are not optional convenience wrappers. zenoh-c's core API functions **require** `z_moved_*_t*` parameters. Without the move functions, these core functions cannot be called:

Listing 2: Exported functions requiring moved parameters

```c
// z_open REQUIRES z_moved_config_t* (line 3796)
ZENOHC_API z_result_t z_open(
   struct z_owned_session_t *this_,
   struct z_moved_config_t *config, // ←requires move
```

```c
    const struct z_open_options_t *_options);

// z_put REQUIRES z_moved_bytes_t* (line 4062)
ZENOHC_API z_result_t z_put(
    const struct z_loaned_session_t *session,
    const struct z_loaned_keyexpr_t *key_expr,
    struct z_moved_bytes_t *payload, // ←requires move
    struct z_put_options_t *options);

// z_config_drop REQUIRES z_moved_config_t* (line 2214)
ZENOHC_API void z_config_drop(struct z_moved_config_t *this_);

// z_declare_subscriber REQUIRES z_moved_closure_sample_t* (line 2331)
ZENOHC_API z_result_t z_declare_subscriber(
    const struct z_loaned_session_t *session,
    struct z_owned_subscriber_t *subscriber,
    const struct z_loaned_keyexpr_t *key_expr,
    struct z_moved_closure_sample_t *callback, // ←requires move
    struct z_subscriber_options_t *options);
```

These functions **are** exported (confirmed via `nm -D`):

```
$ nm -D libzenohc.so | grep " T z_open$"
000000000039acb0 T z_open

$ nm -D libzenohc.so | grep " T z_put$"
000000000039e3f0 T z_put

$ nm -D libzenohc.so | grep " T z_config_drop$"
000000000038f790 T z_config_drop
```

The functions exist. Their required parameter types can only be constructed via functions that do not exist. This is the fundamental barrier.

2.4 What the Moved Types Actually Are

The `z_moved_*_t` types are wrapper structs defined in `zenoh_commons.h`:

```c
typedef struct z_moved_bytes_t {
    struct z_owned_bytes_t _this;
} z_moved_bytes_t;

typedef struct z_moved_config_t {
    struct z_owned_config_t _this;
} z_moved_config_t;
```

At the machine level, `z_moved_bytes_t*` and `z_owned_bytes_t*` point to the same memory — the wrapper struct has the same layout as its single member. The `static inline` move function is an identity cast resolved at compile time with zero runtime cost.

Could Dart perform this cast directly? In theory yes, but:

1. Dart FFI has no mechanism to express the type distinction between `z_owned_bytes_t*` and `z_moved_bytes_t*` — both would be `Pointer<Void>`.
2. `ffigen` v20.1.1 **permanently excludes** `static inline` functions from binding generation (see §1.3).
3. The type safety that the move functions provide (preventing accidental reuse of consumed resources) would be lost entirely.

2.5 How the C Shim Solves This

The shim calls the `static inline` move functions at compile time and exports the result as real functions:

Listing 3: Shim resolves `z_config_move` at compile time

```
FFI_PLUGIN_EXPORT void zd_config_drop(z_owned_config_t* config) {
   z_config_drop(z_config_move(config));
   //         ^^^^^^^^^^^^^^^^^^^^^^^
   //         static inline, resolved at compile time
}

FFI_PLUGIN_EXPORT int zd_open_session(z_owned_session_t* session,
                              z_owned_config_t* config) {
   return z_open(session, z_config_move(config), NULL);
}

FFI_PLUGIN_EXPORT int zd_put(
   const z_loaned_session_t* session,
   const z_loaned_keyexpr_t* keyexpr,
   z_owned_bytes_t* payload) {
   z_put_options_t opts;
   z_put_options_default(&opts);
   return z_put(session, keyexpr, z_bytes_move(payload), &opts);
}
```

The shim applies this pattern consistently across all 8 drop functions:

- `zd_config_drop`, `zd_bytes_drop`, `zd_string_drop`
- `zd_subscriber_drop`, `zd_publisher_drop`, `zd_session_drop`
- `zd_shm_mut_drop`, `zd_shm_provider_drop`

3 Pattern 2: C11 `_Generic` Polymorphic Macros

3.1 The Problem

zenoh-c provides convenience macros that dispatch to the correct type-specific function based on the argument type. These use C11's `_Generic` keyword, a compile-time type-selection mechanism (ISO C11, §6.5.1.1).

From `extern/zenoh-c/include/zenoh_macros.h`:

Listing 4: `z_loan` macro (lines 65–119, 51 branches)

```
#define z_loan(this_) \
   _Generic((this_), \
      z_owned_bytes_t : z_bytes_loan, \
      z_owned_config_t : z_config_loan, \
      z_owned_session_t : z_session_loan, \
      z_owned_publisher_t : z_publisher_loan, \
      z_owned_subscriber_t : z_subscriber_loan, \
      /* ... 40+ more branches ... */ \
   )(&this_)
```

Four such macros exist: `z_loan` (51 branches), `z_drop` (48 branches), `z_move` (56 branches), and `z_loan_mut` (25 branches). None produce symbols:

```
$ nm -D libzenohc.so | grep "z_loan$"
(no output)
```

```
$ nm -D libzenohc.so | grep "z_drop$"
(no output)

$ nm -D libzenohc.so | grep "z_move$"
(no output)
```

3.2 Compounding With Pattern 1

The underlying monomorphic functions (`z_bytes_loan`, `z_config_loan`, etc.) **are** exported. But the `z_move` macro dispatches to the `static inline` move functions (**Pattern 1**). So `z_move(myConfig)` expands to `z_config_move(&myConfig)`, which is `static inline` and not exported.

The C shim bypasses both layers:

```
// Instead of: z_drop(z_move(config))
// The shim does:
z_config_drop(z_config_move(config));
// z_config_drop →exported real function (nm -D confirms)
// z_config_move →static inline, resolved at shim compile time
```

4 Pattern 3: Options Struct Initialization

4.1 The Problem

zenoh-c functions take options structs that must be initialized before use. While the initialization functions (`z_put_options_default`, etc.) are real exported functions, the barrier is the **combination** of:

1. Allocating the options struct with the correct size
2. Initializing it via the default function
3. Setting fields that require move semantics (`opts.encoding = z_encoding_move(...)`)
4. Passing it to the target function (`z_put(session, keyexpr, z_bytes_move(payload), &opts)`)

Steps 3 and 4 require `static inline` functions (**Pattern 1**).

4.2 The Struct Layout Problem

The `z_put_options_t` struct has 8 fields with platform-dependent alignment:

Listing 5: `z_put_options_t` layout from `zenoh_commons.h`

```
typedef struct z_put_options_t {
   struct z_moved_encoding_t *encoding;
   z_congestion_control_t congestion_control;
   z_priority_t priority;
   bool is_express;
   struct z_moved_bytes_t *attachment;
   z_reliability_t reliability;
   const struct z_loaned_timestamp_t *timestamp;
   struct z_moved_source_info_t *source_info;
} z_put_options_t;
```

Dart FFI's Struct class requires explicit field annotations (`@Int32()`, `@Pointer()`) to access struct fields. The `ffigen.yaml` maps all zenoh-c types to Opaque, not `Struct`, because the layout changes between versions.

4.3 How the C Shim Solves This

The shim internalizes the entire options workflow. For functions needing selective overrides, it uses sentinel parameters (`NULL` for strings, `-1` for enums):

Listing 6: Sentinel-based optional parameters in `zd_declare_publisher`

```c
FFI_PLUGIN_EXPORT int zd_declare_publisher(
   const z_loaned_session_t* session,
   z_owned_publisher_t* publisher,
   const z_loaned_keyexpr_t* keyexpr,
   const char* encoding,  // NULL = default
   int congestion_control, // -1 = default
   int priority) {        // -1 = default
   z_publisher_options_t opts;
   z_publisher_options_default(&opts);

   z_owned_encoding_t owned_encoding;
   if (encoding != NULL) {
      z_encoding_from_str(&owned_encoding, encoding);
      opts.encoding = z_encoding_move(&owned_encoding);
   }
   if (congestion_control >= 0) {
      opts.congestion_control = (z_congestion_control_t)congestion_control;
   }
   if (priority >= 0) {
      opts.priority = (z_priority_t)priority;
   }

   return z_declare_publisher(session, publisher, keyexpr, &opts);
}
```

Dart sees simple scalar parameters:

Listing 7: Dart invocation — no options struct, no move functions

```dart
final rc = bindings.zd_declare_publisher(
   loanedSession.cast(),
   ptr.cast(),
   loanedKe.cast(),
   encodingStr.cast(),    // Pointer<Utf8> or nullptr
   congestionControl.index, // int (0 or 1)
   priority.index + 1,    // int (1-7)
);
```

5 Pattern 4: Opaque Type Sizes

5.1 The Problem

zenoh-c types are opaque structs whose sizes are determined at zenoh-c compile time, injected via cbindgen:

Listing 8: Opaque type definitions from `zenoh_opaque.h`

```c
typedef struct ALIGN(8) z_owned_config_t {
 uint8_t _0[2008]; // 2008 bytes
} z_owned_config_t;

typedef struct ALIGN(8) z_owned_session_t {
```

```
  uint8_t _0[8];    // 8 bytes
} z_owned_session_t;

typedef struct ALIGN(8) z_owned_bytes_t {
  uint8_t _0[40];   // 40 bytes
} z_owned_bytes_t;
```

Dart FFI's `sizeOf<T>()` only works for types whose layout is known at Dart compile time. For `Opaque` types, `sizeOf` is not defined.

5.2 How the C Shim Solves This

The shim exports `sizeof` queries as real functions (10 total):

```
FFI_PLUGIN_EXPORT size_t zd_config_sizeof(void) {
   return sizeof(z_owned_config_t); // resolves to 2008
}
FFI_PLUGIN_EXPORT size_t zd_session_sizeof(void) {
   return sizeof(z_owned_session_t); // resolves to 8
}
FFI_PLUGIN_EXPORT size_t zd_bytes_sizeof(void) {
   return sizeof(z_owned_bytes_t); // resolves to 40
}
```

Every Dart wrapper class queries the size at construction time:

```
final size = bindings.zd_session_sizeof(); // query at runtime
final Pointer<Void> ptr = calloc.allocate(size); // allocate exactly
```

This pattern is version-safe: if zenoh-c changes struct sizes in a future release, the Dart code automatically allocates the correct amount.

6 Pattern 5: Closure Callbacks Across Thread Boundaries

6.1 The Problem

zenoh-c delivers asynchronous events via closure structs containing function pointers and a `void*` context:

```
typedef struct z_owned_closure_sample_t {
  void *_context;
  void (*_call)(struct z_loaned_sample_t *sample, void *context);
  void (*_drop)(void *context);
} z_owned_closure_sample_t;
```

The `_call` function pointer is invoked by zenoh-c's internal Tokio runtime threads, not by the Dart isolate's thread.

6.2 Why Dart FFI Cannot Construct These Closures

There are three independent blocking constraints:

6.2.1 Constraint A: No closures in `Pointer.fromFunction`

> *"Creates a Dart function pointer from a top-level or static Dart function. [...] The function must not be a closure (i.e. it must not capture any local variables)."*
> — *Pointer.fromFunction docs*

Dart cannot create a C function pointer that carries state (like which `StreamController` to forward samples to).

6.2.2 Constraint B: Non-Dart thread invocation is undefined behavior

dart-lang/sdk#48865 confirms that calling a `Pointer.fromFunction` callback from a non-Dart thread causes undefined behavior.

6.2.3 Constraint C: `NativeCallable.listener` limitations

Dart 3.1+ added `NativeCallable.listener` which can be called from any thread, but it still cannot capture closures, and context must be passed via integer baton rather than `void*` pointer. See dart-lang/sdk#52689.

6.3 How the C Shim Solves This

The shim implements a **NativePort callback bridge**:

1. **C side:** A static C callback extracts fields from the zenoh-c sample, constructs a `Dart_CObject` array, and posts it to a Dart `ReceivePort` via the thread-safe function `Dart_PostCObject_DL`.
2. **Dart side:** A `ReceivePort` listener on the Dart event loop receives the deserialized data and forwards it to a `StreamController<Sample>`.

Listing 9: NativePort callback bridge (C side)

```c
typedef struct {
  Dart_Port_DL dart_port;
} zd_subscriber_context_t;

static void _zd_sample_callback(z_loaned_sample_t* sample, void* context) {
  zd_subscriber_context_t* ctx = (zd_subscriber_context_t*)context;
  // Extract fields, build Dart_CObject array...
  Dart_PostCObject_DL(ctx->dart_port, &c_array); // thread-safe
}

FFI_PLUGIN_EXPORT int zd_declare_subscriber(
    const z_loaned_session_t* session,
    z_owned_subscriber_t* subscriber,
    const z_loaned_keyexpr_t* keyexpr,
    int64_t dart_port) { // receives port number, not function pointer
  zd_subscriber_context_t* ctx = malloc(sizeof(zd_subscriber_context_t));
  ctx->dart_port = (Dart_Port_DL)dart_port;
  z_owned_closure_sample_t callback;
  z_closure_sample(&callback, _zd_sample_callback, _zd_sample_drop, ctx);
  return z_declare_subscriber(session, subscriber, keyexpr,
                              z_closure_sample_move(&callback), NULL);
}
```

`Dart_PostCObject_DL` is documented as safe to call from any thread, performs deep copy semantics, and requires one-time initialization via `Dart_InitializeApiDL`.

The same pattern is used for all three callback types:

Callback	C function	Data posted	Dart receiver
Subscriber	`_zd_sample_callback`	5-element array	`Subscriber.stream`
Matching	`_zd_matching_status_callback`	Int64 (0/1)	`Publisher.matchingStatus`
Scout	`_zd_scout_hello_callback`	3-element array	`Zenoh.scout()` future

Table 2: NativePort callback bridge usage across the shim

7 Pattern 6: Loaning and Const/Mut Enforcement

7.1 The Problem

zenoh-c implements a **borrowing protocol** where owned types (`z_owned_*_t`) must be "loaned" to produce temporary references (`z_loaned_*_t`) before they can be passed to most API functions. The protocol distinguishes immutable borrows (`const z_loaned_*_t*`) from mutable borrows (`z_loaned_*_t*`).

Listing 10: Loan function pair from zenoh_commons.h

```
// zenoh_commons.h:2218-2222
ZENOHC_API const struct z_loaned_config_t *z_config_loan(
   const struct z_owned_config_t *this_); // immutable borrow
ZENOHC_API struct z_loaned_config_t *z_config_loan_mut(
   struct z_owned_config_t *this_);       // mutable borrow
```

7.2 Why the Per-Type Functions ARE Exported

Unlike Pattern 1 (`static inline` move functions), the per-type loan functions **are real, exported symbols** in `libzenohc.so`:

```
$ nm -D libzenohc.so | grep -E 'z_(config|session|bytes|publisher)_loan'
000000000038ad40 T z_bytes_loan
000000000038ad50 T z_bytes_loan_mut
0000000000390370 T z_config_loan
0000000000390380 T z_config_loan_mut
000000000039e810 T z_publisher_loan
00000000003a59b0 T z_session_loan
```

This is critical: these functions could theoretically be called from Dart FFI via `DynamicLibrary.lookup`. However, two independent barriers prevent direct use.

7.3 Barrier A: Macro Dispatch

The zenoh-c API presents loaning through `_Generic` macros (**Pattern 2**). No symbol named `z_loan` or `z_loan_mut` exists in `libzenohc.so`. Dart must know which monomorphic function to call for each type—a responsibility the shim absorbs.

7.4 Barrier B: Const Qualification Erasure

Even if Dart called the per-type functions directly, Dart FFI's type system **erases the const/mut distinction**. All zenoh-c types are mapped to `ffi.Opaque` in `ffigen.yaml`. A `Pointer<z_loaned_config_t>` in Dart carries no information about whether the pointer is `const` or mutable.

> *"A Pointer represents a pointer into native C memory."*
> — *Pointer class (Dart 3.11.1)*

There is no `ConstPointer<T>` in `dart:ffi`. This means:

- Dart cannot distinguish `const z_loaned_config_t*` from `z_loaned_config_t*`
- There is no compile-time or runtime check preventing mutable operations on an immutably-borrowed reference
- Incorrect usage would cause undefined behavior in zenoh-c

7.5 How the Shim Resolves This

1. **Macro bypass:** Calls monomorphic functions directly (`z_config_loan` not `z_loan(config)`)
2. **Const enforcement in C:** Function signatures enforce const correctness at the C compiler level
3. **Naming convention:** `zd_*_loan` for immutable borrows, `zd_*_loan_mut` for mutable borrows

Listing 11: Loan wrappers enforce const at the C level

```
// Returns const pointer -- C compiler prevents mutation
FFI_PLUGIN_EXPORT const z_loaned_config_t*
   zd_config_loan(const z_owned_config_t* config);

// Returns mutable pointer -- allows mutation (SHM buffer writes)
FFI_PLUGIN_EXPORT z_loaned_shm_mut_t*
   zd_shm_mut_loan_mut(z_owned_shm_mut_t* buf);
```

7.6 Why This Is Distinct From Patterns 1 and 2

Aspect	Pattern 1	Pattern 2	Pattern 6
Barrier	No exported symbol (`static inline`)	No exported symbol (`_Generic` macro)	Exported symbol exists, but const/mut qualifier is lost across FFI
`nm -D`	Function absent	Macro absent	**Function present** (`T` in symbol table)
Could Dart call it?	No	No	Technically yes, but **unsafely**
What the shim adds	Inlines the call into exported body	Calls monomorphic function directly	Enforces const/mut at C level + provides stable naming

Table 3: Pattern 6 is a semantic barrier, not a syntactic one

8 End-to-End Worked Example: `z_put`

This section traces a single operation — publishing data on a key expression — through the six patterns to show how they compound.

8.1 What a C Consumer Does

```
z_put_options_t opts;
z_put_options_default(&opts);        // Pattern 3

z_owned_bytes_t payload;
z_bytes_copy_from_str(&payload, "Hello");

z_put(z_session_loan(&session),      // Pattern 6 + Pattern 2
     z_view_keyexpr_loan(&keyexpr),  // Pattern 6 + Pattern 2
     z_bytes_move(&payload),         // Pattern 1 (static inline)
     &opts);                         // Pattern 3
```

8.2 What Dart FFI Cannot Do

Dart FFI **can** call `z_put` (exported), `z_put_options_default` (exported), and `z_bytes_copy_from_str` (exported).

Dart FFI **cannot** call: `z_bytes_move` (`static inline`, **Pattern 1**), `sizeof(z_put_options_t)` (**Pattern 4**), or set `opts.encoding` (requires `z_encoding_move`, **Pattern 3**).

8.3 The Full Call Chain

```
Dart: session.put('demo/test', 'Hello')
  -> ZBytes.fromString('Hello')
    -> calloc.allocate(bindings.zd_bytes_sizeof()) [Pattern 4]
    -> bindings.zd_bytes_copy_from_str(ptr, nativeStr) [exported]
  -> bindings.zd_session_loan(sessionPtr)    [Pattern 6]
  -> bindings.zd_view_keyexpr_loan(kePtr)    [Pattern 6]
  -> bindings.zd_put(loanedSession, loanedKe, payloadPtr) [shim]
    -> C: z_put_options_default(&opts)      [Pattern 3]
    -> C: z_put(session, keyexpr,
             z_bytes_move(payload),         [Pattern 1]
             &opts)                         [Pattern 3]
      -> libzenohc.so: z_put()              [exported]
  -> payload.markConsumed()                 [Dart ownership]
```

> Four patterns, one operation. The C shim collapses all of them into a single FFI-callable function with simple scalar parameters.

8.4 What the Shim Does NOT Do

The shim is deliberately thin. It does not add error recovery, buffer data, manage threads, add abstraction layers, or change zenoh-c's ownership semantics. Every `zd_*` function maps to one or a small sequence of `z_*` calls. The shim is a **mechanical flattening layer**, not an abstraction.

Part II
Function-by-Function Analysis

9 Initialization (2 functions)

Function	Wraps	Purpose
`zd_init_dart_api_dl`	`Dart_InitializeApiDL`	Initializes Dart native API for `Dart_PostCObject_DL`
`zd_init_log`	`zc_init_log_from_env_or`	Initializes zenoh logger

Assessment: Correct and minimal. Called once during `_initBindings()`.

10 Config (5 functions)

Function	Wraps	Patterns
`zd_config_sizeof`	`sizeof(z_owned_config_t)`	P4
`zd_config_default`	`z_config_default`	P3
`zd_config_insert_json5`	`z_config_loan_mut` + `zc_config_insert_json5`	P1, P2
`zd_config_loan`	`z_config_loan`	P1, P2
`zd_config_drop`	`z_config_drop(z_config_move(...))`	P1

11 Session (4 functions)

Function	Wraps	Patterns
`zd_session_sizeof`	`sizeof(z_owned_session_t)`	P4
`zd_open_session`	`z_open(session, z_config_move(config), NULL)`	P1, P3
`zd_session_loan`	`z_session_loan`	P1, P2
`zd_close_session`	`z_close` + `z_session_drop`	P1

Finding F1: `z_close` return value ignored

The return value of `z_close` is discarded in `zd_close_session`. zenoh-cpp similarly ignores it. Close-time errors are non-actionable. **Severity: Low.**

12 KeyExpr (4), Bytes (6), String/View String (7)

All are straightforward **Pattern 1** + **Pattern 4** wrappers. Key points:

- View key expressions borrow the backing string; lifetime is guaranteed by `_withKeyExpr`'s `try/finally`.
- `zd_string_data` returns non-null-terminated data; Dart correctly uses `toDartString(length: len)` everywhere.
- View string convenience functions (`zd_view_string_data/len`) combine loan + access in one FFI round-trip.

13 Put / Delete (2 functions)

Both initialize options to defaults and pass through. `zd_put` moves the payload; Dart calls `payload.markConsumed()` after. Session-level `zd_put` intentionally omits encoding/QoS/attachment — those are available via `zd_publisher_put`.

14 Subscriber (3 functions + 2 static callbacks)

Resolves Patterns 1, 4, and 5. The subscriber callback is the primary motivation for **Pattern 5**. Context is heap-allocated via `malloc`, freed by `_zd_sample_drop` when the subscriber is undeclared. Error-path closure drop is defensively correct.

15 Publisher (8 functions + 2 static callbacks)

Resolves all five patterns. Both payload and attachment are moved (consumed) in `zd_publisher_put`. The matching listener uses the NativePort bridge (**Pattern 5**).

16 Info / ZID (4 functions)

The two ZID collection functions (`zd_info_routers_zid`, `zd_info_peers_zid`) use synchronous buffer-based collection (stack-allocated context, `NULL` drop callback) instead of NativePort—appropriate because these are synchronous. Buffer holds up to 64 ZIDs (1024 bytes).

17 Scout (2 functions + 1 static callback)

Uses NativePort with **stack-allocated** context (safe because `z_scout` blocks until timeout). Posts null sentinel on completion; Dart `Completer` resolves on null.

Finding F6: Scout return value ignored

`Zenoh.scout()` does not check the return value from `zd_scout`. Scout failures are indistinguishable from "no entities found." **Severity: Medium.**

18 Shared Memory (13 functions, conditionally compiled)

All guarded by:

```
#if defined(Z_FEATURE_SHARED_MEMORY)
&& defined(Z_FEATURE_UNSTABLE_API)
```

Alloc results are flattened from a compound result struct to return code + out-parameter. Dart returns `null` on allocation failure (not throwing).

Part III
Cross-Cutting Analysis

19 Memory Ownership Protocol

The Dart side implements a three-state model mirroring zenoh-c's gravestone semantics:

State	Flags	Operations
Live	`_disposed=false, _consumed=false`	All operations
Consumed	`_consumed=true`	None (`StateError`)
Disposed	`_disposed=true`	`dispose()` (no-op)

Two-step cleanup is applied consistently: (1) `zd_*_drop()` releases zenoh-c internals (Rust allocator), then (2) `calloc.free()` releases the Dart-allocated wrapper memory.

20 Thread Safety

All C callbacks use only stack-local variables + read-only context access + `Dart_PostCObject_DL` (thread-safe). No shared mutable state. Context structs contain only a `Dart_Port_DL` integer, written once during declaration and read from callbacks—no races possible.

21 Findings

F1: `z_close` return value ignored

Severity: Low. zenoh-cpp similarly ignores it. Close-time errors are non-actionable.

F2: Payload passes through `z_bytes_to_string`

Severity: Low. An unnecessary copy in the subscriber callback. Sending raw bytes directly from `z_sample_payload` would be more efficient but requires the more complex bytes reader API.

F3: `ZBytes.fromUint8List` element-by-element copy

Severity: Low. Could use `nativeBuf.asTypedList(len).setAll(0, data)` for bulk copy via `memcpy`. The SHM path already uses this pattern correctly.

F4: `z_encoding_from_str` return code unchecked

Severity: Low. In practice, zenoh-c's `z_encoding_from_str` always succeeds (treats the string as an opaque MIME type). Defensive fix would be to check the return code.

F5: Allocator mismatch in `Zenoh.initLog`

Severity: Low. `toNativeUtf8()` allocates via `malloc` but cleanup uses `calloc.free()`. Both resolve to the same system `free()` in practice, but violates the matching-allocator convention.

F6: Scout return value ignored

Severity: Medium. `Zenoh.scout()` does not check the return value from `zd_scout`. Scout failures are indistinguishable from "no entities found."

F7: `ZBytes.dispose()` leaks calloc wrapper when consumed

Severity: Low. When consumed, `dispose()` returns early without calling `calloc.free(_ptr)`. The native content was consumed by zenoh-c, but the calloc wrapper still needs freeing. `ShmMutBuffer.dispose()` handles this correctly.

Part IV
Pattern Resolution Matrix

This matrix maps every C shim function to which patterns it resolves.

C shim function	P1	P2	P3	P4	P5	P6
zd_init_dart_api_dl					•	
zd_init_log						
zd_config_sizeof				•		
zd_config_default			•			
zd_config_insert_json5	•	•				
zd_config_loan		•				•
zd_config_drop	•					
zd_session_sizeof				•		
zd_open_session	•		•			
zd_session_loan		•				•
zd_close_session	•	•				
zd_view_keyexpr_sizeof				•		
zd_view_keyexpr_from_str	•					
zd_view_keyexpr_loan		•				•
zd_keyexpr_as_view_string	•					
zd_bytes_sizeof				•		
zd_bytes_copy_from_str	•					
zd_bytes_copy_from_buf	•					
zd_bytes_to_string	•					
zd_bytes_loan		•				•
zd_bytes_drop	•					
zd_string_sizeof				•		
zd_string_loan		•				•
zd_string_data	•					
zd_string_len	•					
zd_string_drop	•					
zd_view_string_sizeof				•		
zd_view_string_data	•	•				
zd_view_string_len	•	•				
zd_put	•		•			
zd_delete			•			
zd_subscriber_sizeof				•		
zd_declare_subscriber	•				•	
zd_subscriber_drop	•					
zd_publisher_sizeof				•		
zd_declare_publisher	•		•			
zd_publisher_loan		•				•
zd_publisher_drop	•					
zd_publisher_put	•		•			
zd_publisher_delete			•			
zd_publisher_keyexpr	•					
zd_pub.._matching_listener	•				•	
zd_pub.._matching_status						
zd_info_zid	•					
zd_id_to_string						
zd_info_routers_zid					•	
zd_info_peers_zid					•	
zd_scout	•		•		•	
zd_whatami_to_view_string						
zd_shm_provider_sizeof				•		

C shim function	P1	P2	P3	P4	P5	P6
`zd_shm_provider_new`	•					
`zd_shm_provider_loan`		•				•
`zd_shm_provider_drop`	•					
`zd_shm_provider_available`						
`zd_shm_mut_sizeof`				•		
`zd_shm_provider_alloc`	•					
`zd_shm..alloc_gc_defrag..`	•					
`zd_shm_mut_loan_mut`		•				•
`zd_shm_mut_data_mut`						
`zd_shm_mut_len`						
`zd_bytes_from_shm_mut`	•					
`zd_shm_mut_drop`	•					
Totals	**34**	**14**	**8**	**12**	**6**	**8**

Key: P1 = `static inline` functions, P2 = `_Generic` macro dispatch, P3 = options structs, P4 = opaque `sizeof`, P5 = closure callbacks, P6 = loaning (const/mut enforcement).

Note: Loan functions are no longer marked P1. The per-type loan functions (`z_config_loan`, `z_bytes_loan`, etc.) are exported symbols in `libzenohc.so`—they are not `static inline`. P6 marks the 8 explicit loan wrappers whose primary purpose is const/mut enforcement and macro bypass for the borrowing protocol.

Part V
Conformance Summary

22 C Best Practices

Practice	Status	Notes
All exports use `FFI_PLUGIN_EXPORT`	PASS	`visibility("default")`
`C_VISIBILITY_PRESET hidden`	PASS	Only marked symbols visible
Consistent `zd_` namespace prefix	PASS	No collisions with `z_`/`zc_`
Return code checking	PARTIAL	F4: `z_encoding_from_str`
Memory cleanup on all paths	PASS	Closures dropped on failure
No undefined behavior	PASS	All pointer arithmetic bounded
Const correctness	PASS	`const` on loan params
Feature guards for optional functionality	PASS	SHM behind `#ifdef`

23 Dart FFI Best Practices

Practice	Status	Notes
Lazy singleton for bindings	PASS	Initialized on first access
`Dart_InitializeApiDL` before NativePort	PASS	Called in `_initBindings`
`ReceivePort` for async callbacks	PASS	Subscriber, matching, scout
`calloc.allocate(sizeof)` pattern	PASS	Consistent across all types
Ownership tracking	PASS	All wrapper classes
`try/finally` for temp allocations	PASS	`_withKeyExpr`, encoding
Idempotent `close()/dispose()`	PASS	All classes
Explicit length for strings	PASS	`toDartString(length:)`

24 Architecture Conformance

Principle	Status	Notes
Shim is mechanical (no business logic)	PASS	1–5 zenoh-c calls per function
Single-load library pattern	PASS	OS resolves `libzenohc.so`
Options structs internalized	PASS	Dart never touches `z_*_options_t`
Move semantics handled in C	PASS	Dart never calls `z_*_move`
Callback bridge via NativePort	PASS	Thread-safe, copies data
Sentinel values for optional params	PASS	NULL for strings, -1 for enums

Part VI
Pattern Summary

Pat.	What It Is	Why Dart Cannot Call It	How the Shim Resolves It
1	56 `static inline` move functions	`static` = internal linkage; `inline` = no symbol. `nm -D` confirms absence. ffigen permanently excludes them.	Shim calls them at compile time. C compiler inlines the cast into the exported function.
2	4 `_Generic` macros (25–56 branches)	Preprocessor construct. No symbol. Dispatches to type-specific functions at compile time.	Shim calls monomorphic functions directly.
3	Options structs with fields requiring P1	`sizeof` unknown. Fields require `z_*_move`. Layout is version-specific.	Shim allocates, initializes, sets fields, and passes—all in one exported function.
4	Opaque type sizes (8–2008 bytes)	Dart FFI has no `sizeof()` for foreign types. Types mapped to `Opaque`.	Shim exports `zd_*_sizeof()` functions. Dart queries at runtime.
5	Closure callbacks on Tokio threads	`Pointer.fromFunction` cannot capture state. Non-Dart thread = UB.	NativePort bridge: C callback posts `Dart_CObject` via `Dart_PostCObject_DL`.
6	Loaning protocol (const/mut borrowing)	Per-type functions **are** exported (`nm -D` confirms), but `z_loan`/`z_loan_mut` macros are not (P2). Dart's `Pointer<Opaque>` erases const qualifier.	Shim wraps each loan with explicit naming (`zd_*_loan` vs `zd_*_loan_mut`). Const enforced at C compiler level.

Table 5: Six FFI barrier patterns and their C shim resolutions

The six patterns are not independent barriers — they **compound**. A single `z_put` call requires Pattern 4 (to allocate the payload), Pattern 6 (to loan the session and key expression), Pattern 1 (to move the payload), and Pattern 3 (to create and pass options). A subscriber declaration adds Pattern 5 (callback bridge). The C shim collapses all of these into single FFI-callable functions with simple scalar parameters.

Appendices

A Findings Summary

ID	Severity	Description
F1	Low	`z_close` return value ignored in `zd_close_session`
F2	Low	Payload passes through `z_bytes_to_string` in subscriber callback
F3	Low	`ZBytes.fromUint8List` element-by-element copy
F4	Low	`z_encoding_from_str` return code unchecked
F5	Low	`Zenoh.initLog` frees `malloc`'d string with `calloc.free`
F6	Medium	`Zenoh.scout` ignores `zd_scout` return value
F7	Low	`ZBytes.dispose()` leaks calloc wrapper when consumed

Table 6: All findings

B References

B.1 Dart FFI Official Documentation

All `api.dart.dev/stable/latest/` URLs resolve to the current stable SDK (Dart 3.11.1 at time of writing). Using `/latest/` ensures links remain valid as new Dart versions are released.

Document	URL
dart:ffi overview	https://api.dart.dev/stable/latest/dart-ffi/dart-ffi-library.html
C interop guide	https://dart.dev/interop/c-interop
DynamicLibrary class	https://api.dart.dev/stable/latest/dart-ffi/DynamicLibrary-class.html
DynamicLibrary.lookup	https://api.dart.dev/stable/latest/dart-ffi/DynamicLibrary/lookup.html
Pointer.fromFunction	https://api.dart.dev/stable/latest/dart-ffi/Pointer/fromFunction.html
Opaque class	https://api.dart.dev/stable/latest/dart-ffi/Opaque-class.html
Struct class	https://api.dart.dev/stable/latest/dart-ffi/Struct-class.html
sizeOf function	https://api.dart.dev/stable/latest/dart-ffi/sizeOf.html
NativeApi.initializeApiDLData	https://api.dart.dev/stable/latest/dart-ffi/NativeApi/initializeApiDLData.html
ffigen package	https://pub.dev/packages/ffigen

B.2 Bug Tracker Citations

Issue	Status	Key Finding
ffigen#146	Closed	ffigen now **skips** inline functions
native#459	Partial	`static inline` permanently excluded
sdk#48865	Open	Non-Dart thread callback = undefined behavior
sdk#52689	Open	`NativeCallable.listener` limitations

B.3 zenoh-c Source Files

File	Content	Lines
`zenoh_macros.h`	`static inline` moves, `_Generic`, C++ overloads	1467
`zenoh_commons.h`	Exported function signatures, struct typedefs	~5000
`zenoh_opaque.h`	Opaque type definitions with compile-time sizes	~800
`zenoh_dart.h`	C shim header — 62 exported functions	543
`zenoh_dart.c`	C shim implementation	~700

B.4 ISO C Standards

Standard	Section	Content
ISO C11	§6.5.1.1	`_Generic` selection — compile-time type dispatch
ISO C11	§6.7.4	`inline` function specifiers
ISO C11	§6.2.2	`static` linkage — internal to translation unit

B.5 Symbol Table Verification

Commands run against `extern/zenoh-c/target/release/libzenohc.so` (zenoh-c v1.7.2):

```
# Verify static inline functions are NOT exported
nm -D libzenohc.so | grep z_bytes_move # (no output)
nm -D libzenohc.so | grep z_config_move # (no output)

# Verify _Generic macros are NOT exported
nm -D libzenohc.so | grep "z_loan$" # (no output)
nm -D libzenohc.so | grep "z_drop$" # (no output)

# Verify real functions ARE exported
nm -D libzenohc.so | grep " T z_put$"
  000000000039e3f0 T z_put
nm -D libzenohc.so | grep " T z_open$"
  000000000039acb0 T z_open
nm -D libzenohc.so | grep " T z_config_drop$"
  000000000038f790 T z_config_drop
```